

# Macros in C++

1. what is a macro or pre processor in c++ programming language. .... 2



## 1. what is a macro or pre processor in c++ programming language

In **C++**, a **macro** or **preprocessor directive** is a command that is processed **before the actual compilation** of your code begins.

## Preprocessor Overview

When you write a C++ program, it first goes through a **preprocessing phase** — handled by the **C++ preprocessor**.

The preprocessor performs tasks like:

- Including header files (**#include**)
- Defining constants or macros (**#define**)
- Conditional compilation (**#if**, **#ifdef**, etc.)

All preprocessor commands **begin with #** and are not actual C++ statements — they don't end with a semicolon.

## What is a Macro

A **macro** is a **named piece of code** (or constant) that gets **replaced by its value or code block** before the compiler sees it.

You define it using the **#define** directive.

## Types of Macros

### 1. Object-like Macros (Simple constants)

```
cpp

#define PI 3.14159

int main() {
    double area = PI * 5 * 5;
    // Before compilation, PI is replaced with 3.14159
}
```

### 2. Function-like Macros

```
cpp

#define SQUARE(x) ((x) * (x))

int main() {
    int result = SQUARE(5);    // becomes ((5) * (5))
}
```

 **Note:** Macros don't have type checking — the preprocessor just replaces text. For example:

```
cpp

SQUARE(3+2)    // becomes ((3+2)*(3+2)) = 25 (correct)
```

But if written poorly:

```
cpp

#define BAD_SQUARE(x) (x*x)
BAD_SQUARE(3+2)    // becomes (3+2*3+2) = 11 (WRONG!)
```

## Common Preprocessor Directives

| Directive                        | Description                                        |
|----------------------------------|----------------------------------------------------|
| <b>#include</b>                  | Adds contents of another file (e.g., header files) |
| <b>#define</b>                   | Defines a macro constant or function               |
| <b>#undef</b>                    | Undefines a macro                                  |
| <b>#ifdef / #ifndef</b>          | Checks if a macro is defined or not                |
| <b>#if, #elif, #else, #endif</b> | Conditional compilation                            |

### Example:

```
cpp

#include <iostream>
#define DEBUG 1
#define SQUARE(x) ((x)*(x))

int main() {
    #if DEBUG
        std::cout << "Debug mode ON\n";
    #endif

    std::cout << "Square of 5 is " << SQUARE(5) << std::endl;
```

```
    return 0;  
}
```

If you set `#define DEBUG 0`, the debug line won't print — demonstrating **conditional compilation**.

---

Would you like me to show how macros compare to **inline functions** (which are often safer replacements in modern C++)?